

GRC Project ER to Relational Mapping

Department(**dept_id**, dept_name)

PK: dept_id

This table stores one record for each department in the organization. It includes a unique department ID and the name of the department. The primary key is dept_id.

Employee(**employee_id**, employee_name, job_title, dept_id)

PK: employee_id

FK: dept_id references Department(dept_id)

This table stores one record for each employee. It includes the employee's unique ID, name, job title, and the department to which the employee belongs. Each employee must belong to exactly one department, so dept_id is a foreign key that references the Department table and cannot be null. The primary key is employee_id.

Risk(**risk_id**, risk_title, risk_level, risk_status, employee_id)

PK: risk_id

FK: employee_id references Employee(employee_id)

This table stores one record for each risk. It includes the risk's unique ID, title, level, status, and the employee who owns the risk. Each risk must have exactly one employee owner, so employee_id is a foreign key that references the Employee table and cannot be null. The primary key is risk_id.

Control(**control_id**, control_name, control_status, employee_id)

PK: control_id

FK: employee_id references Employee(employee_id)

This table stores one record for each control. It includes the control's unique ID, name, status, and the employee who owns the control. Each control must have exactly one employee owner, so employee_id is a foreign key that references the Employee table and cannot be null. The primary key is control_id.

CompReq(**req_id**, frw_name, req_code)

PK: req_id

This table stores one record for each compliance requirement. It includes a unique requirement ID, the framework name, and the requirement code. A compliance requirement may be satisfied by zero or many controls. The primary key is req_id.

Audit(**audit_id**, audit_name, audit_date)

PK: audit_id

This table stores one record for each audit. It includes the audit's unique ID, name, and date. An audit may evaluate zero or many controls. The primary key is audit_id.

Mitigates(**control_id, risk_id**)

PK: (risk_id, control_id)

FK: risk_id references Risk(risk_id)

FK: control_id references Control(control_id)

This table represents the many-to-many relationship between Risk and Control. It stores which controls mitigate which risks. A risk may be mitigated by zero or many controls, and a control may mitigate zero or many risks. The composite primary key is (risk_id, control_id).

Satisfies(**control_id, req_id**)

PK: (control_id, req_id)

FK: control_id references Control(control_id)

FK: req_id references CompReq(req_id)

This table represents the many-to-many relationship between Control and Compliance Requirement. It stores which controls satisfy which compliance requirements. A control may satisfy zero or many compliance requirements, and a compliance requirement may be satisfied by zero or many controls. The composite primary key is (control_id, req_id).

Evaluates(**control_id, audit_id**, finding_status)

PK: (audit_id, control_id)

FK: audit_id references Audit(audit_id)

FK: control_id references Control(control_id)

This table represents the many-to-many relationship between Audit and Control. It stores which controls are evaluated in which audits. It also stores finding_status, which records the result of the evaluation for a specific audit-control pair. The composite primary key is (audit_id, control_id).